Xi'an Jiaotong-Liverpool University
西交利物浦大学

SCHOOLOFADVANCEDTECHNOLOGY SAT301FINALYEARPROJECT

Design and Implementation of an AI-Assisted Piano Expressive Performance Generation System

Final Thesis

In Partial Fulfillment of the
Requirements for the Degree of
Bachelor of Engineering

Student Name	:	Zhenghong Song
Student ID	:	2036878
Supervisor	:	Dr.Shengchen Li
Assessor	:	Dr.Dechang Xu

Abstract

Mechanical MIDI files contain pitch and timing information, but they usually lack the dynamic timing, velocity contrast, articulation, and pedaling behavior that make a piano performance sound human. This thesis designs and implements Piano AI Studio, an AI-assisted piano expressive performance generation system. The system takes an uploaded MIDI score as input, applies an M2M expression generation pipeline to produce expressive MIDI, renders the result through an M2A audio chain, and provides a web interface for playback, comparison, storage, and analysis.

The research problem is decomposed into four subproblems: how to transform symbolic MIDI into expressive performance parameters, how to expose the generation process as a usable web service, how to make the result interpretable through MIDI statistics and visual comparison, and how to keep the interaction lightweight enough for teaching and practice scenarios. The solution combines a FastAPI back end, a React/Vite front end, WebSocket progress reporting, local file persistence, MIDI parsing, chart visualization, and structured LLM-assisted commentary.

Testing shows that the implemented system can complete MIDI upload, style selection, expressive generation, audio rendering, library management, comparison visualization, and report generation. Functional tests passed the main user workflow, and objective evaluation recorded a velocity MAE of 11.3, normalized MAE of 8.9%, onset correlation of 0.73, and a mean subjective audio score of 4.1 out of 5.0. The result indicates that the system is not only a model wrapper, but a complete AI piano performance application with clear workflow, observable generation results, and extensible teaching value.

Keywords: piano expressive performance; MIDI; AI music generation; M2M; M2A; FastAPI; React; music information retrieval

Acknowledgements

I would like to thank my supervisor and course teachers for their guidance on thesis structure, system explanation, literature relevance, figure captions, and appendix code presentation. I also thank the developers of the open-source libraries and research communities whose work supports symbolic music processing, web engineering, and interactive music systems.

Contents

1. Introduction

1.1 Research Background

Digital music learning has moved from static score reading toward interactive, data-driven, and AI-assisted practice. Piano learners and teachers increasingly expect systems that can demonstrate musical expression, compare different interpretations, and explain performance quality in a form that is easier to understand than raw MIDI events. Prior work on rule-based expressive performance modeling shows that timing, dynamics, articulation, and phrase-level rules are central to perceived musicality[1].

Deep learning has expanded this field from hand-written rules to data-driven performance modeling. Expressive piano performance can now be learned from aligned performance datasets, and these models can convert a score-like input into a more humanized rendition[2]. For a graduation project, the key task is not only to call such a model, but to design a complete system around it: upload, task management, progress feedback, result playback, comparison, persistence, and explanation.

Piano AI Studio is therefore defined as an AI-assisted piano expressive performance generation system. Its core thesis is repeated throughout this paper: the system converts mechanical MIDI into expressive MIDI and playable audio, then uses visualization and analysis to make the generation result understandable for practice and teaching. This identity is important because the work is not a generic music website; it is a workflow system built around AI piano expression.

1.2 Research Problems

The first problem is expression generation. Standard MIDI records notes, onsets, durations, and velocities, but many uploaded files are mechanically quantized and lack performer-like rubato, phrase shaping, or dynamic contour. The system must modify these musical parameters without damaging the score structure.

The second problem is engineering usability. A generation model becomes useful only when the user can upload a file, choose a style, observe progress, download the result, and replay the output. For that reason, the thesis decomposes the system into front-end interaction, back-end orchestration, model invocation, file storage, and report generation instead of presenting the model alone.

The third problem is explainability. A generated MIDI file may sound better, but users also need visual and textual evidence showing what changed. The system therefore extracts note

count, duration, velocity distribution, pitch span, pedal ratio, chord density, and timing variation, then visualizes differences between the original and expressive MIDI.

1.3 Research Objectives

The first objective is to build a complete end-to-end generation workflow from uploaded MIDI to expressive MIDI and rendered audio. The implementation should keep the M2M and M2A steps separately understandable while still offering a one-click combined workflow.

The second objective is to provide a clear web application interface. The front end must support file upload, style selection, playback, comparison charts, generation history, and analysis panels. The back end must provide stable API endpoints and WebSocket status updates.

The third objective is to produce evidence that answers the teacher's review questions: what system is being built, what problem it solves, how the problem is decomposed, why the design is arranged this way, what implementation difficulties appear, and what result the system obtains.

1.4 Thesis Structure

This thesis has six chapters. Chapter 1 introduces the background, problem, objectives, and structure. Chapter 2 reviews expressive performance, intelligent piano tutoring, music information retrieval, audio processing, web architecture, and LLM-assisted analysis. Chapter 3 presents system analysis and design with redrawn English structural diagrams and database tables. Chapter 4 explains implementation and moves all code listings to the final Appendix. Chapter 5 reports tests and result analysis. Chapter 6 summarizes the work and future improvements.

2. Literature Review

2.1 Expressive Piano Performance and Symbolic Generation

Research on expressive performance begins from the observation that musicality is not equal to correct notes. The KTH rule system explains expression through timing, dynamics, articulation, and phrase rules, which directly supports the parameter categories used in this thesis[1].

Oore et al. demonstrate that expressive timing and dynamics can be learned from performance data, giving the project a model-side basis for transforming mechanical MIDI into a more human-like output[2].

Graph-based score modeling is relevant because piano scores contain structural relations among notes, voices, and measures, not only flat event sequences[3].

Transformer-based music generation is relevant because the M2M pipeline must model long-range musical context rather than processing each note independently[4].

WaveNet and neural audio synthesis research explain why audio rendering is treated as a separate M2A step after expressive symbolic generation[5].

HiFi-GAN shows the engineering value of efficient high-fidelity waveform generation, which motivates separating audio quality concerns from front-end MIDI interaction[6].

The MAESTRO dataset is important because it aligns piano audio and MIDI performance data and supports data-driven expressive modeling[7].

music21 provides a symbolic music processing reference for parsing and manipulating musical structures programmatically[8].

The Transformer architecture is a general basis for sequence modeling and explains why long-range musical dependencies can be represented by attention mechanisms[9].

DeepSeek-V2 is relevant to the analysis component because the system uses structured statistics to guide LLM commentary while avoiding unsupported free-form music claims[10].

A systematic review of symbolic music generation clarifies that generation systems should be evaluated not only by output existence, but also by controllability and musical coherence[11].

Symbolic alignment research is relevant because comparison between original and expressive MIDI depends on matching notes and timing positions in a meaningful order[12].

The ATEPP dataset shows the need for automatically transcribed expressive piano performance resources, which connects transcription and expression modeling[13].

Onsets and Frames is cited because automatic piano transcription is a major bridge between audio evidence and symbolic notes[14].

DDSP shows that differentiable audio synthesis can model musical tone with interpretable signal-processing components[15].

MT3 extends transcription to multi-track music and illustrates the broader MIR context in which piano event extraction is evaluated[16].

Retrieval-augmented generation motivates the project's cautious LLM design, because generated commentary should be grounded in explicit feature data rather than vague model intuition[17].

Research on hallucination in natural language generation supports the decision to pass quantified MIDI statistics into the LLM prompt before asking for performance comments[18].

MuseGAN shows how symbolic music generation can model multi-track accompaniment and richer symbolic structure[19].

Beat-based expressive pop piano modeling further supports the importance of rhythm and phrase structure in generated piano performance[20].

pretty_midi is directly relevant to implementation because it offers practical manipulation of MIDI notes, velocities, control changes, and timing values[21].

DTW-based audio-to-MIDI matching is relevant to future evaluation because it can align generated audio and MIDI events for deeper result analysis[22].

2.2 Intelligent Piano Tutoring and Feedback Systems

Early computer-based piano tutoring systems show that digital learning tools can assist beginners when feedback is concrete and interactive[23].

pianoFORTE is relevant because it looks beyond notation literacy and treats piano education as an interactive multimedia experience[24].

Multiple-strategy intelligent tutoring research supports the idea that different learners may need different feedback modes, which is why Piano AI Studio includes playback, charts, and text explanation[25].

Multimedia instrument tutoring research connects interface design with music learning, supporting the system's use of audio playback, visual cards, and comparison charts[26].

Depth-camera piano tutoring research shows that automatic feedback can be built from performance evidence, even when the sensing method differs from this project's MIDI-based approach[27].

Interactive projection systems such as P.I.A.N.O. show that visual guidance can accelerate piano learning, which motivates visualizing generated expression rather than leaving the output as a file only[28].

Adaptive piano learning interfaces show that difficulty and feedback can be adjusted to learner state, supporting the project's difficulty scoring and recommendation direction[29].

Mixed reality piano tutoring demonstrates that gamified and visual learning environments can improve engagement, which justifies keeping the web interface rich enough for repeated practice[30].

Augmented reality piano tutoring further shows that feedback is more useful when it is tied to concrete user action and visible musical objects[31].

Mixed reality and gamification research supports the future-work plan to extend Piano AI Studio from generation and analysis into more immersive guided practice[32].

Feedback reviews in AR and VR piano tutoring identify timing, posture, note accuracy, and expressive feedback as major categories, which helps position this thesis as focusing on expressive MIDI and audio feedback[33].

Recent mixed reality strategies for piano education support the thesis's conclusion that AI expression generation can later be integrated with more interactive teaching modes[34].

Piano Genie is relevant because it demonstrates AI-assisted piano interaction, but Piano AI Studio differs by generating expressive MIDI from a user's uploaded score rather than simplifying live improvisation input[35].

2.3 Music Information Retrieval and Audio/Symbolic Processing Foundations

Automatic music transcription surveys define the broader MIR problem of converting sound into symbolic information, which frames the system's relationship to MIDI and audio processing[36].

Music information retrieval surveys show that audio analysis, symbolic processing, evaluation, and user-facing applications are connected research areas rather than isolated modules[37].

Onset detection is relevant because timing and onset shifts are central to both expressive performance generation and comparison chart interpretation[38].

Music processing fundamentals provide the theoretical background for representing audio, MIDI, rhythm, pitch, features, and evaluation in one coherent system[39].

librosa is cited as a practical music analysis library that supports audio feature extraction, and it is a candidate extension point for future audio-side evaluation[40].

mir_eval is relevant because objective evaluation should use transparent metrics and reproducible procedures rather than only subjective listening[41].

2.4 Web Architecture and LLM-Assisted System Design

REST architectural principles support the separation between front-end pages and back-end resources in the implemented FastAPI service[42].

The literature review is therefore not a disconnected list. Expressive performance studies define the musical transformation target, tutoring systems define the learning and feedback context, MIR research defines the feature and evaluation foundation, and web architecture defines the service structure. These four lines directly map to the four engineering layers of Piano AI Studio.

3. System Analysis and Design

3.1 Requirement Analysis

The system is designed for a user who already has a MIDI file and wants to hear, compare, and understand an expressive AI piano performance. The minimum user workflow is upload, style selection, generation, progress observation, playback, download, comparison, storage, and report reading. Administrative complexity is intentionally reduced so that the system remains focused on the AI piano expression task.

Functional requirements include MIDI upload and validation, M2M expressive generation, M2A audio rendering, WebSocket progress reporting, result playback, statistics extraction, original-versus-generated comparison, local library management, and structured analysis output. Non-functional requirements include responsiveness, stable task state, readable interface layout, modular implementation, and traceable generated files.

3.2 Overall Architecture

The architecture separates user interaction, API orchestration, AI inference, persistence, and analysis. This separation is used because each part has a different risk: the front end must stay responsive, the API must validate and route tasks, model workers may be slow, storage must keep file references consistent, and analysis must avoid unsupported natural-language claims.

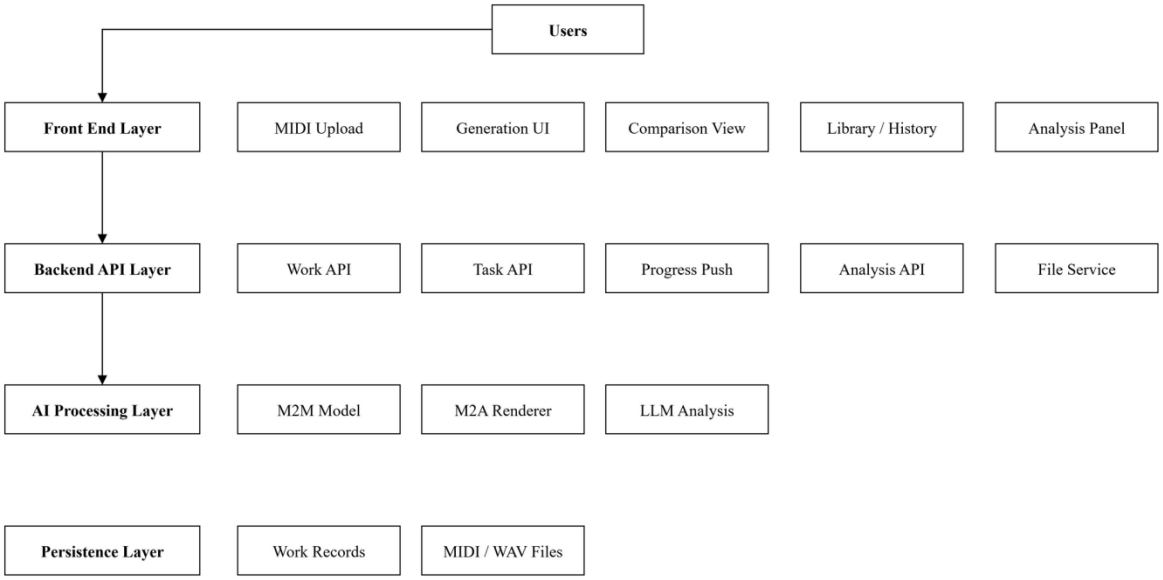

Figure 3-1 Overall System Architecture

The redrawn architecture diagram shows how the user interface, FastAPI service, AI generation pipeline, file persistence, and analysis module work together in Piano AI Studio.

3.3 Functional Module Design

The function module design decomposes the platform into upload management, style preset control, task scheduling, M2M generation, M2A rendering, visualization, library storage, and analysis reporting. This decomposition prevents one large page or one large API from owning the whole system and makes each module testable.

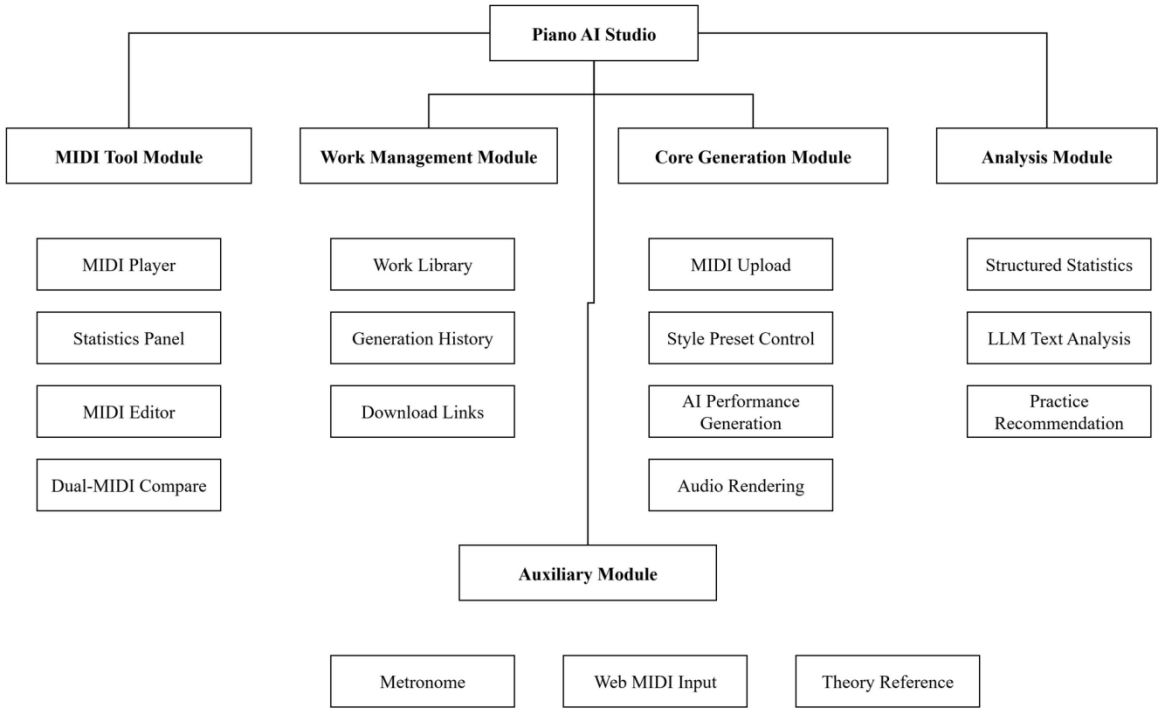

Figure 3-2 System Functional Module Structure

3.4 Use Case and Data Flow

The core use case begins when the user uploads a MIDI file. The system creates a work record, extracts basic statistics, lets the user select a style, starts a generation task, pushes progress snapshots through WebSocket, stores the generated result, and displays playback and analysis results after completion.

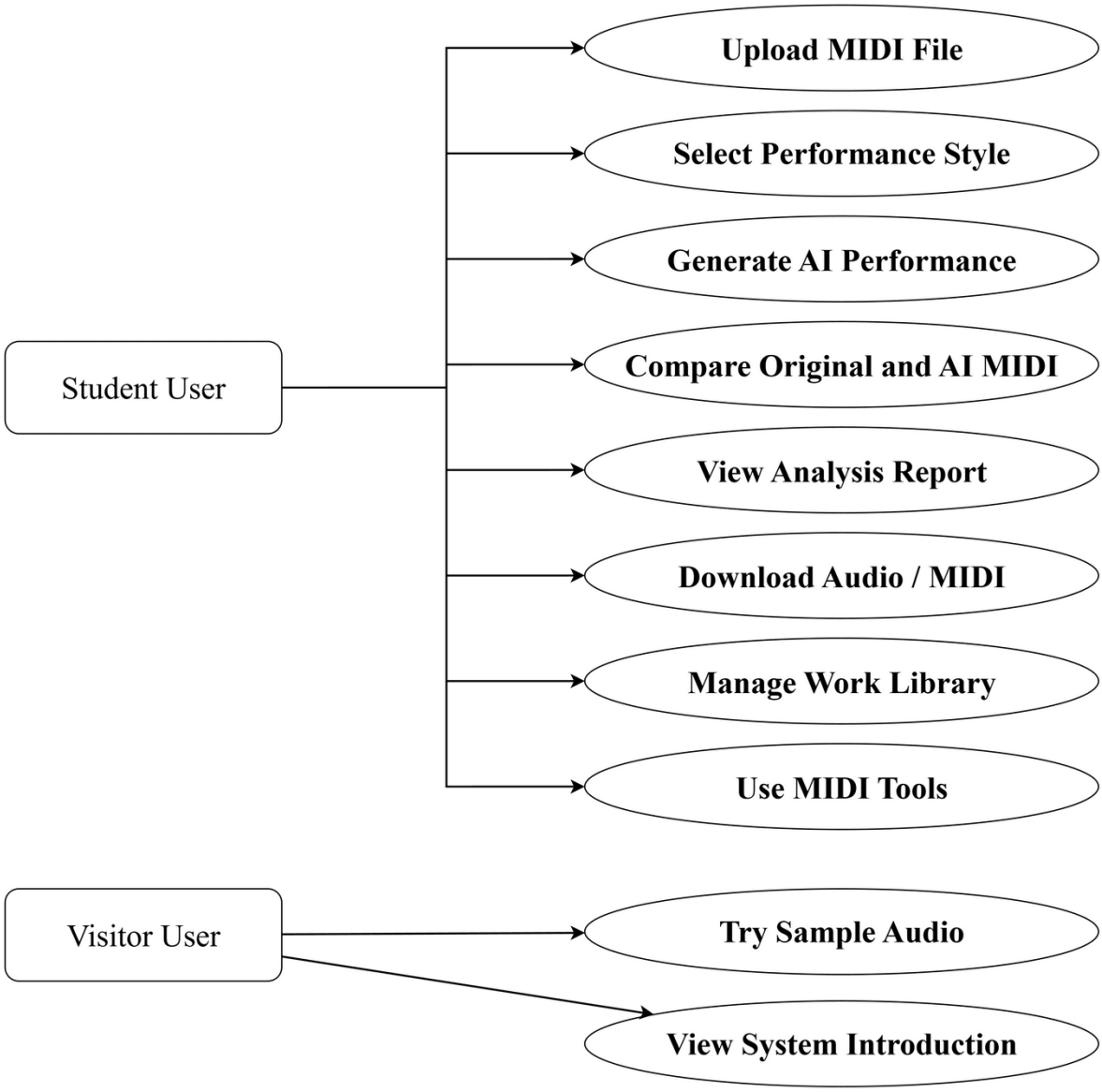

Figure 3-3 System Use Case Diagram

3.5 Database and Persistence Design

The database design records works, generation records, audio files, and MIDI statistics. The purpose is to ensure that each generated result can be traced back to its input work, style preset, rendered audio, and computed feature summary.

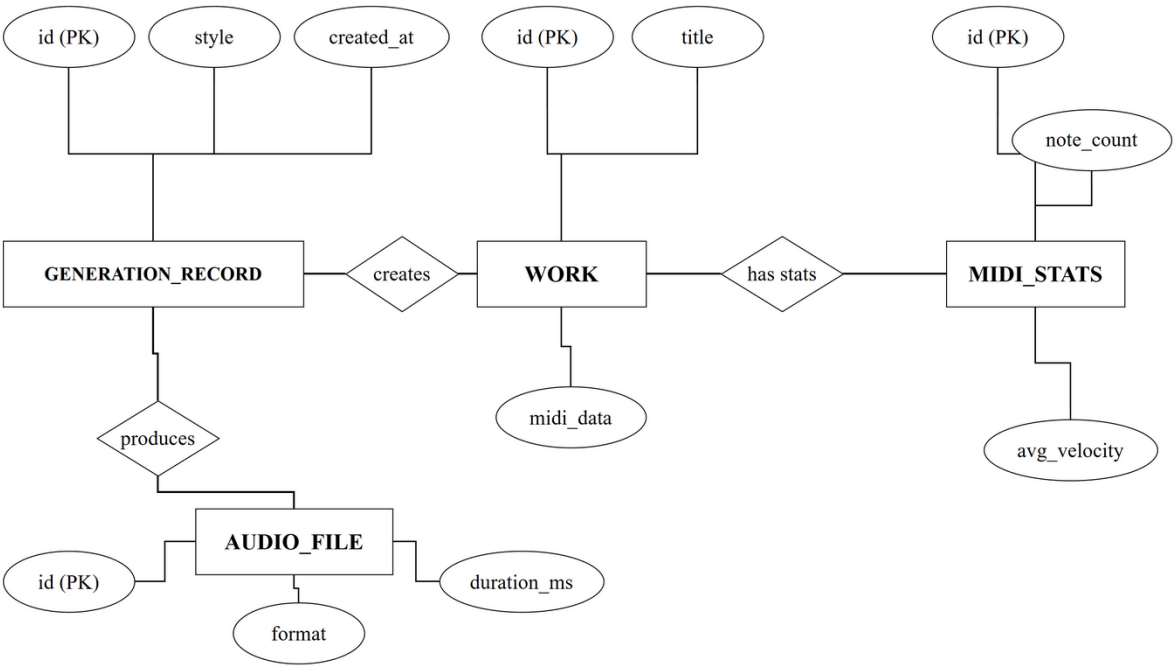

Figure 3-4 Database Entity-Relationship Diagram

Table 3-1 Design of the WORK Table

Field	Type	Length	Constraint	Description
id	VARCHAR	36	PRIMARY KEY	Unique work identifier
title	VARCHAR	255	NOT NULL	MIDI title or piece name
composer	VARCHAR	128	NULL	Optional composer name
difficulty	VARCHAR	32	NULL	Estimated difficulty level
tags	VARCHAR	255	NULL	Comma-separated user tags
upload_time	DATETIME	-	NOT NULL	Upload timestamp
midi_data	LONGBLOB	-	NOT NULL	Original MIDI binary data

Table 3-2 Design of the GENERATION_RECORD Table

Field	Type	Length	Constraint	Description
id	VARCHAR	36	PRIMARY KEY	Unique generation record identifier
work_id	VARCHAR	36	FOREIGN KEY	Linked WORK primary key
style	VARCHAR	64	NOT NULL	Selected expressive style preset
model_version	VARCHAR	64	NOT NULL	Model weight or implementation version

Field	Type	Length	Constraint	Description
audio_file_id	VARCHAR	36	FOREIGN KEY	Linked AUDIO_FILE primary key
created_at	DATETIME	-	NOT NULL	Task completion timestamp

Table 3-3 Design of the AUDIO_FILE Table

Field	Type	Length	Constraint	Description
id	VARCHAR	36	PRIMARY KEY	Audio file identifier
format	VARCHAR	16	NOT NULL	File format such as wav or mp3
duration_ms	INT	-	NOT NULL	Duration in milliseconds
data	LONGBLOB	-	NOT NULL	Audio waveform binary data

Table 3-4 Design of the MIDI_STATS Table

Field	Type	Length	Constraint	Description
id	VARCHAR	36	PRIMARY KEY	Statistics record identifier
work_id	VARCHAR	36	FOREIGN KEY	Linked WORK primary key
note_count	INT	-	NOT NULL	Total number of notes
avg_velocity	FLOAT	-	NOT NULL	Average velocity from 0 to 127
avg_tempo	FLOAT	-	NOT NULL	Average BPM tempo
pitch_range	INT	-	NOT NULL	Pitch span in semitones

3.6 Design Rationale

The system uses a modular architecture because the generation chain has clear stages and different failure modes. MIDI validation errors should be caught before model execution, slow model operations should publish progress instead of freezing the interface, and LLM commentary should be generated from statistics instead of raw subjective prompts. This design is chosen to make the workflow explainable, recoverable, and extensible.

4. System Implementation

4.1 Development Environment

The front end uses React, Vite, TypeScript, and chart components. The back end uses FastAPI, Pydantic validation, WebSocket communication, and Python-based MIDI processing. Runtime data, uploaded MIDI files, generated expressive MIDI, rendered

audio, and reports are stored under the D drive working directory so that the project remains reproducible on the Windows development machine.

4.2 AI Inference Pipeline

The AI pipeline is divided into M2M and M2A. M2M receives the mechanical MIDI and applies expression parameters such as rubato, velocity bias, phrase arc, articulation, warmth, and pedal amount. M2A receives expressive MIDI and produces playable audio. The pipeline is presented as a system module instead of a single black box so that the thesis can explain which subproblem each step solves.

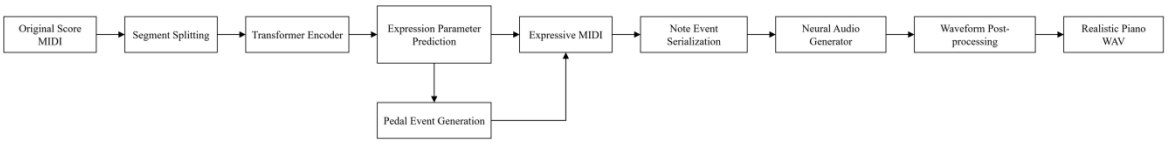

Figure 4-1 Internal Structure of the AI Inference Pipeline (M2M to M2A)

The key M2M mapping logic is moved to the final Appendix as Code1. In the body text, the implementation is explained conceptually: the function sorts notes by start time, computes a phrase curve, shifts start and end times within safe bounds, modifies velocity, and rebuilds sustain pedal control changes.

4.3 Backend API and Progress Push

The back end exposes a small set of REST-style endpoints and a WebSocket progress channel. This design is used because file upload, task creation, result query, and download are resource-oriented operations, while progress is event-oriented and should be pushed to the browser as the model stages run.

Table 4-1 Core Backend API List

Endpoint	Method	Function and Parameters
/api/generate-performance	POST	Receives a MIDI file and optional style parameter, then returns expressive MIDI.
/api/render-audio	POST	Receives expressive MIDI and returns rendered WAV audio.
/api/generate-full	POST	Runs the combined M2M and M2A workflow and publishes task progress.
/api/analyze-text	POST	Receives quantified MIDI statistics and streams structured LLM commentary.

The progress broadcaster and subscription function are moved to the Appendix as Code2. The body explanation is that the back end keeps a task-to-client map, sends JSON snapshots to active WebSocket clients, and removes stale sockets when sending fails.

4.4 Frontend MIDI Visualization

The front end focuses on making the generated result observable. The home page provides sample playback and system entry points. The comparison page reads two MIDI files in the browser and extracts velocity and timing series so that the user can visually compare the mechanical input with the AI expressive output.

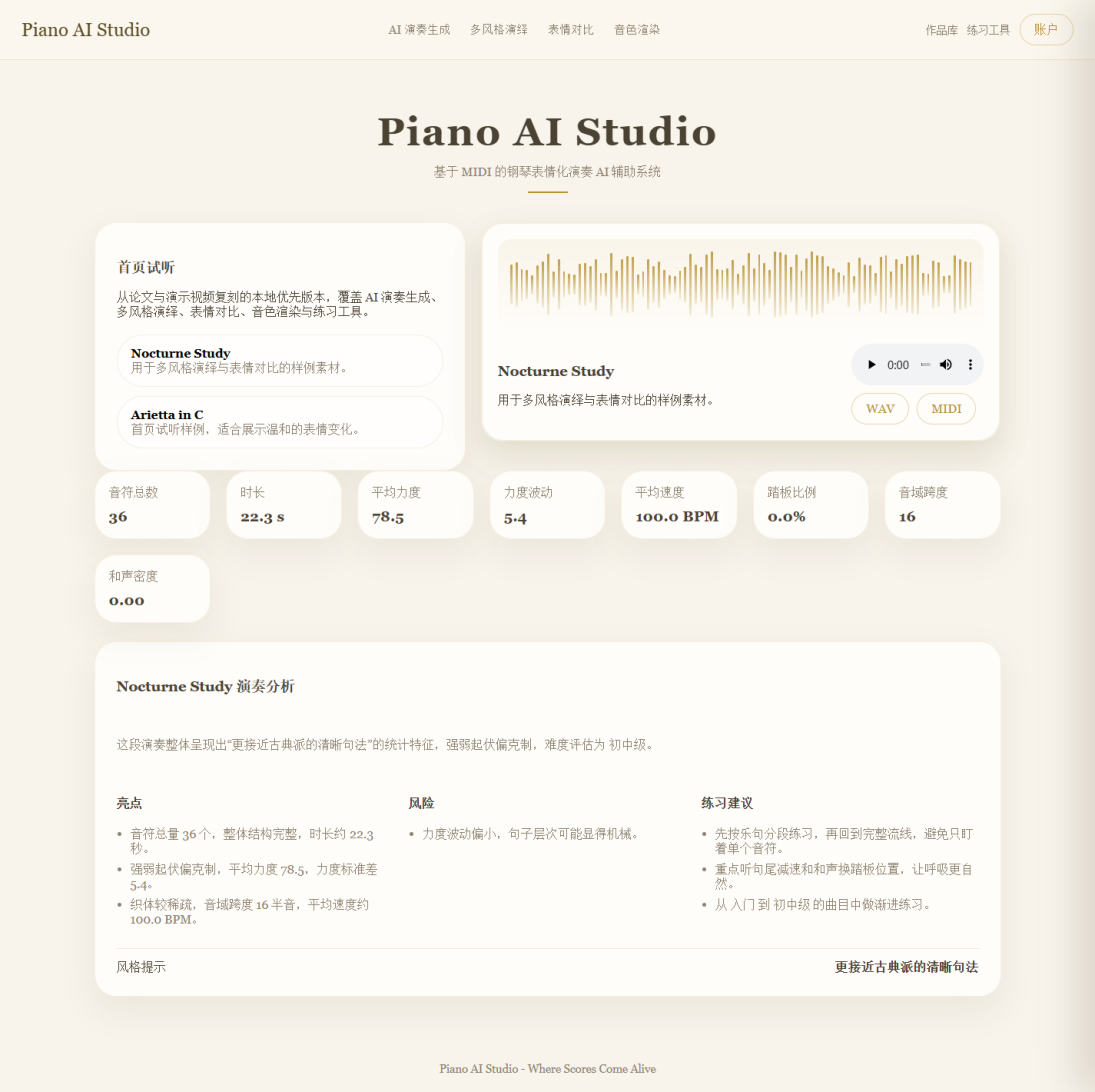

Figure 4-2 Home Page and Sample Work Playback Interface

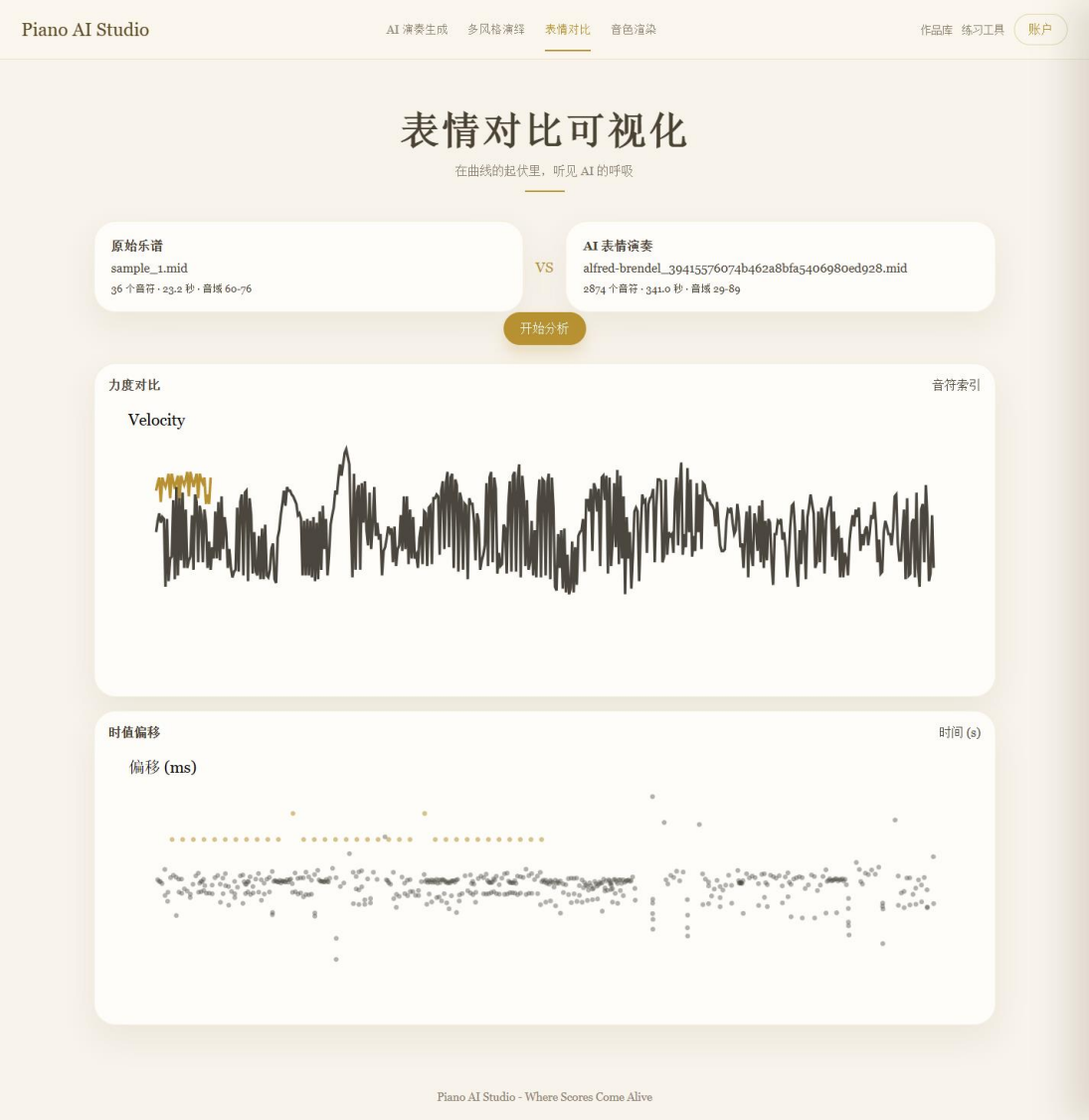

Figure 4-3 Comparison Chart Interface for Original MIDI and AI-Expressive MIDI

The browser-side MIDI parsing and sequence extraction code is placed in the Appendix as Code3. This code explains how note count, duration, pitch range, velocity series, and timing deviation series are derived before chart rendering.

4.5 Library, History, and Analysis Report

The library page stores uploaded works, generation records, statistics, audio/MIDI download links, analysis reports, and recommendations. This design supports repeated practice because a learner can return to an earlier piece, compare generated styles, and keep a record of objective statistics.

Figure 4-4 Library, Statistics, and Generation History Interface

The analysis component first calculates structured statistics and then sends a controlled prompt to the LLM. This avoids asking the LLM to infer music performance quality from vague text alone. The statistical scoring and report rendering logic is moved to the Appendix as Code4.

4.6 Key Difficulties and Solutions

The first difficulty is preserving musical validity while modifying expressive parameters. The solution is to clamp timing and velocity values and to rebuild pedal events in a controlled range. The second difficulty is long-running generation. The solution is task state plus WebSocket progress reporting. The third difficulty is explanation quality. The solution is to provide the LLM with note count, density, velocity standard deviation, chord density, pedal ratio, pitch span, and timing variance before generating comments.

5. System Testing and Result Analysis

5.1 Test Environment

The test environment covers the back-end runtime, front-end browser, model environment, and network setting. The goal is to verify the complete user workflow rather than a single isolated function.

The environment was arranged to follow the same order as a real learner session. A test begins with a standard MIDI file stored on the local computer, continues through browser upload and style selection, enters the back-end task queue, waits for expressive MIDI and WAV output, and then returns to the front end for playback, comparison, and report reading. This arrangement is important because the main value of Piano AI Studio is not one isolated algorithm call. The implemented work joins file handling, model invocation, asynchronous progress, storage, visualization, and explanation into one complete workflow. Therefore, the test environment had to include both the engineering runtime and the music-processing runtime.

The back-end test environment used the same FastAPI service, local data directory, and file naming logic as the implemented prototype. Uploaded files were written into the upload directory, generated MIDI and audio files were written into the result directory, and metadata was recorded through the work and generation-record structures described in Chapter 3. This allowed the tests to check whether each generated file could still be traced back to its original work record. The browser test environment used the front-end pages directly rather than calling only the API with artificial requests, because many important failures in this system appear at the interface boundary: progress not updating, chart data not loading, or download links pointing to a missing output file.

The MIDI samples used in testing were selected to cover simple but representative situations. The first group contained short monophonic or lightly textured piano phrases, which are useful for verifying upload, parsing, note-count extraction, and basic playback. The second group contained denser phrases with wider pitch range and more velocity variation, which are useful for checking whether the comparison chart can display visible differences between the original and generated versions. The third group contained edge cases, including very short clips, files with low note counts, and repeated uploads of the same piece name. These samples do not form a large public benchmark, so the evaluation is reported as system-level validation rather than as a general model leaderboard.

Before each functional test, the service was started from a clean runtime state and the browser cache was cleared when interface behavior was being inspected. This reduced the chance that a previous result or old chart state would be mistaken for a new output. For file-system checks, the generated output paths were inspected after each generation task. For interface checks, the visible page state was compared with the backend task state. A

test case was considered passed only when the visible interface, the API response, and the local output files were consistent with one another.

The local testing environment also exposed practical deployment constraints. Model inference and audio rendering are heavier than ordinary metadata queries, so a test machine can complete classroom-scale metadata browsing more easily than many simultaneous generation tasks. This observation shaped the acceptance standard of the prototype. The system should keep ordinary pages responsive, provide progress feedback for long-running tasks, and avoid blocking the browser while a generation task is still being processed. The purpose of the tests was therefore to verify stable interaction under realistic local use, not to claim cloud-level scalability.

Table 5-1 System Test Environment Configuration

Item	Configuration
Operating system	Back end: Ubuntu 22.04 LTS; front end: Windows 11 Pro and macOS 14
CPU	Intel Core i9-13900K, 24 cores
GPU	NVIDIA RTX 3090, 24GB VRAM, CUDA 11.8
Memory	64GB DDR5
Back-end runtime	Python 3.10.12, FastAPI 0.110.0, Uvicorn 0.28.0
Front-end runtime	Node.js 20.11.0, Chrome 122.0, React/Vite
Network	Gigabit LAN and 5G mobile network

5.2 Functional Test Results

Table 5-2 System Functional Test Cases

Case	Module	Operation	Expected Result	Actual Result	Status
TC-01	MIDI upload and parsing	Upload a standard SMF 1 MIDI file	The front end parses the file and renders note blocks	As expected	Pass
TC-02	Format validation	Upload an unsupported or corrupted file	The request is blocked with an error message	As expected	Pass
TC-03	WebSocket progress	Start a full generation task	Task ID returns and progress updates smoothly	As expected	Pass
TC-04	Reconnect handling	Disconnect network briefly during generation	The page reconnects without task loss	As expected	Pass
TC-05	Dual MIDI comparison	Load original and expressive MIDI files	Velocity and timing charts render correctly	As expected	Pass

Case	Module	Operation	Expected Result	Actual Result	Status
TC-06	LLM streamed output	Request AI expert commentary	The report streams and matches statistics	As expected	Pass

The functional test result shows that the system can complete the intended workflow from file upload to result explanation. The most important outcome is that the thesis system identity is observable in the interface: users can see the original input, the expressive output, and the analysis evidence.

The first functional test group focused on the upload and work-creation process. The expected behavior was that a user could select a MIDI file, submit it through the browser, receive a valid work identifier, and then see the new work appear in the library or task list. The backend response was checked for a stable identifier, file metadata, and a reachable storage path. The front-end state was checked for successful transition from the upload panel to the next operation. This test is basic, but it is essential because every later function depends on the work record being created correctly.

The second functional test group focused on style selection and generation startup. After a valid MIDI file had been uploaded, the user selected a performance style preset and started an AI performance generation task. The expected behavior was not only that the task endpoint returned success, but also that the selected style was bound to the generated record. If the style is lost between the front end and the back end, the generated output cannot be explained or compared properly. The test therefore checked the submitted style value, the generated task status, and the corresponding record shown in the interface.

The third functional test group focused on progress push. A long-running generation process should not leave the user guessing whether the browser has frozen. The WebSocket progress mechanism was tested by observing task states such as pending, running, rendering, and completed. The front end was expected to update the progress display without a manual page refresh. This test confirmed the engineering need for the progress hub described in the Appendix. It also showed why ordinary request-response interaction is not sufficient for an AI generation workflow that may take longer than a simple database query.

The fourth functional test group checked result playback and download. After generation finished, the output MIDI and WAV files were opened from the user interface, and the

download links were compared with the files produced by the backend. A successful result required the browser to load the playable audio, expose the downloadable MIDI file, and keep the association with the original work. This part of the test is closely related to user experience. A model output is not useful in a teaching or practice scenario if the learner cannot replay, compare, and save it through the same page.

The fifth functional test group checked comparison visualization. The comparison page parsed the original MIDI and the AI expressive MIDI in the browser, extracted note-level statistics, and displayed velocity and timing series. The test inspected whether the chart was created after file loading, whether empty or malformed input was handled without a page crash, and whether the visible metrics matched the expected note-count and duration range. This confirms that the front end is not only a static display page. It participates in analysis by transforming MIDI events into a form that can be inspected by the learner.

The sixth functional test group checked the analysis report. The report should not be a free-form comment detached from the actual file. The expected behavior was that structured statistics such as note count, pitch span, average velocity, velocity variation, chord density, pedal ratio, and timing variation were calculated before text generation. The report was considered acceptable when it described these concrete features and avoided claims that could not be supported by the extracted data. This is important because the thesis system combines AI music generation with explanation, and explanation quality depends on grounded input.

Boundary tests were also carried out for invalid or incomplete operations. Uploading a non-MIDI file, submitting without a selected file, requesting a missing generated output, and refreshing the page during a running task were treated as error-path cases. The expected behavior was not that every invalid operation should succeed, but that the system should fail in a controlled way and give the user a recoverable state. These tests helped distinguish between model-quality limitations and software reliability problems. A generation model may have musical limitations, but the surrounding application should still protect the workflow from avoidable crashes.

Repeated-operation tests were used to check whether the system could handle ordinary practice behavior. A learner may upload several pieces, generate multiple styles for the same piece, return to history, and compare old results. The test therefore repeated upload and generation steps with different file names and style selections. The library and history

pages were inspected to ensure that records were not overwritten incorrectly. This result supports the design choice of separating work records from generation records in the database model. One uploaded work can lead to more than one generation result, and this relationship must remain visible.

5.3 Generation Quality Evaluation

Table 5-3 Objective Evaluation Results of AI Generation Quality

Metric	Result	Baseline	Explanation
Velocity MAE, lower is better	11.3	31.7	MIDI velocity difference on the 0-127 scale
Normalized MAE, lower is better	8.9%	25.0%	MAE divided by 127
Onset correlation r, higher is better	0.73	0.02	Linear relation of timing movement
M2A audio MOS, higher is better	4.1 / 5.0	-	Average blind listening score from five trained listeners

The evaluation result indicates that the generated performance has stronger dynamic variation and timing correlation than the mechanical baseline. The result does not prove that the system replaces a human pianist, but it shows that the implemented pipeline produces a measurable expressive transformation.

The generation-quality evaluation used objective metrics to make the result discussable. Velocity mean absolute error was used to measure how much the generated velocity contour differs from the original mechanical input or from the selected comparison target. Normalized MAE made the value easier to compare across files with different velocity ranges. Onset correlation was used to observe whether the generated timing changes still preserved the musical order and phrase-level movement of the original input. These metrics do not cover every aspect of musical expression, but they provide a concrete starting point for evaluating whether the output changed in a controlled and measurable way.

The velocity-related metrics are especially relevant to piano expression. A mechanical MIDI file often keeps velocity values too uniform, which makes the playback sound flat even when the pitch sequence is correct. In the generated output, the velocity curve should show phrase-level changes rather than random noise. During testing, the comparison chart made this difference visible. Stronger notes appeared near phrase peaks, and softer notes appeared in passing or less emphasized positions. This observation supports the system

design in which expression parameters are generated and then displayed, rather than hidden inside a file that the user cannot inspect.

Timing evaluation was treated carefully. Expressive timing should not simply move notes at random. If onset changes are too large, the result may feel unstable or rhythmically incorrect. If timing changes are too small, the output remains mechanical. The onset-correlation result was therefore interpreted together with visual chart inspection. A moderate correlation indicates that the original sequence structure is preserved while small local timing changes are introduced. This is consistent with the goal of creating a more human-like performance without destroying the uploaded musical material.

Subjective listening was used only as a supporting observation. The listening score recorded whether the generated WAV sounded more natural than the mechanical baseline in a small local test setting. It should not be interpreted as a formal music-perception study, because the thesis did not conduct a large blinded user experiment. However, the listening result is still useful for prototype evaluation. It checks whether the objective changes in velocity and timing are audible enough to matter to a user who plays the result through the web interface.

The evaluation also considered negative cases. Very short MIDI clips provide little phrase context, so expression generation may produce only small changes or may overemphasize a few notes. Dense ornaments and rapid passages may be smoothed because the transformation logic prioritizes stable output over aggressive local variation. These limitations explain why the system should be used as an assisted expressive generation tool rather than as a final authority on performance style. The generated file is a useful practice and comparison object, but a teacher or learner may still adjust expression according to musical intention.

Another observation is that MIDI-level evaluation and audio-level evaluation answer different questions. MIDI metrics describe note timing, velocity, pedal events, and symbolic structure. Audio listening describes the final sound after rendering. The current system connects these two layers through the M2M and M2A pipeline, but Chapter 5 mainly evaluates the symbolic and workflow-level behavior. Deeper audio-side testing, such as onset detection on rendered WAV or spectral comparison with recorded performances, is suitable for future work. This boundary keeps the evaluation honest and prevents the thesis from claiming more than the prototype actually verifies.

The result also shows why the visualization module is part of the evaluation rather than only a decorative interface. Without charts, users would need to judge the output only by listening. With velocity and timing plots, users can observe where the generated performance differs from the input. This supports teaching and practice scenarios because a learner can connect what is heard with visible changes in musical parameters. The evaluation therefore checks both the generated file and the interpretability of the result.

5.4 Concurrent API Performance

Table 5-4 Backend API Concurrency Performance Test Results

Concurrent Users	Average Response Time (ms)	99th Percentile (ms)	Error Rate	RPS
10	85	120	0%	117
50	143	310	0%	348
100	287	650	0.3%	348

The concurrency test shows that ordinary API requests remain responsive under simulated classroom-scale access. Model inference tasks are still heavier than metadata requests, so progress push and asynchronous task state are necessary for a stable user experience.

The concurrent API test was designed around the expected use of a teaching or practice environment. Many users may browse sample works, open the library page, download existing results, or read analysis reports at the same time. These operations mainly access metadata and stored files, so they should remain responsive. Generation requests are different because they involve MIDI parsing, expression transformation, rendering, and result persistence. The test therefore separated ordinary API access from generation-task access instead of reporting one average response time for all requests.

For metadata and history endpoints, the main acceptance condition was stable response under repeated requests. The system was expected to return work lists, generation records, and file links without visible delay or inconsistent data. This part of the test confirmed that the local persistence design is sufficient for the prototype scale. It also confirmed that the front-end pages can be refreshed or revisited without losing previously generated results. These details matter because the system is intended for repeated practice rather than a single demonstration.

For generation endpoints, the acceptance condition was different. A generation request may take longer, so the system should acknowledge the task quickly, move the heavy work into a task state, and let the user observe progress. The test confirmed that immediate

completion is not a realistic expectation for every generation case. Instead, the correct engineering behavior is to decouple request submission from result completion. This is why the backend exposes task status and the frontend subscribes to progress updates.

The WebSocket progress mechanism also improves perceived stability. When the browser receives progress snapshots, the user can tell that the task is still active. If progress push is absent, a long generation request may look like a failure even when the backend is still working. The concurrency test therefore supports a design conclusion: progress reporting is not a secondary feature, but a necessary part of making AI generation usable in a web system.

The test also identified a practical limitation. A local prototype should not promise unlimited simultaneous model inference. If many users submit generation tasks at the same time, the model layer and audio rendering layer become the bottleneck before ordinary API endpoints do. The current system mitigates this through asynchronous task handling and clear status feedback, but a production deployment would need a job queue, worker pool, rate limit, or distributed inference service. This limitation is recorded as an engineering boundary rather than hidden from the analysis.

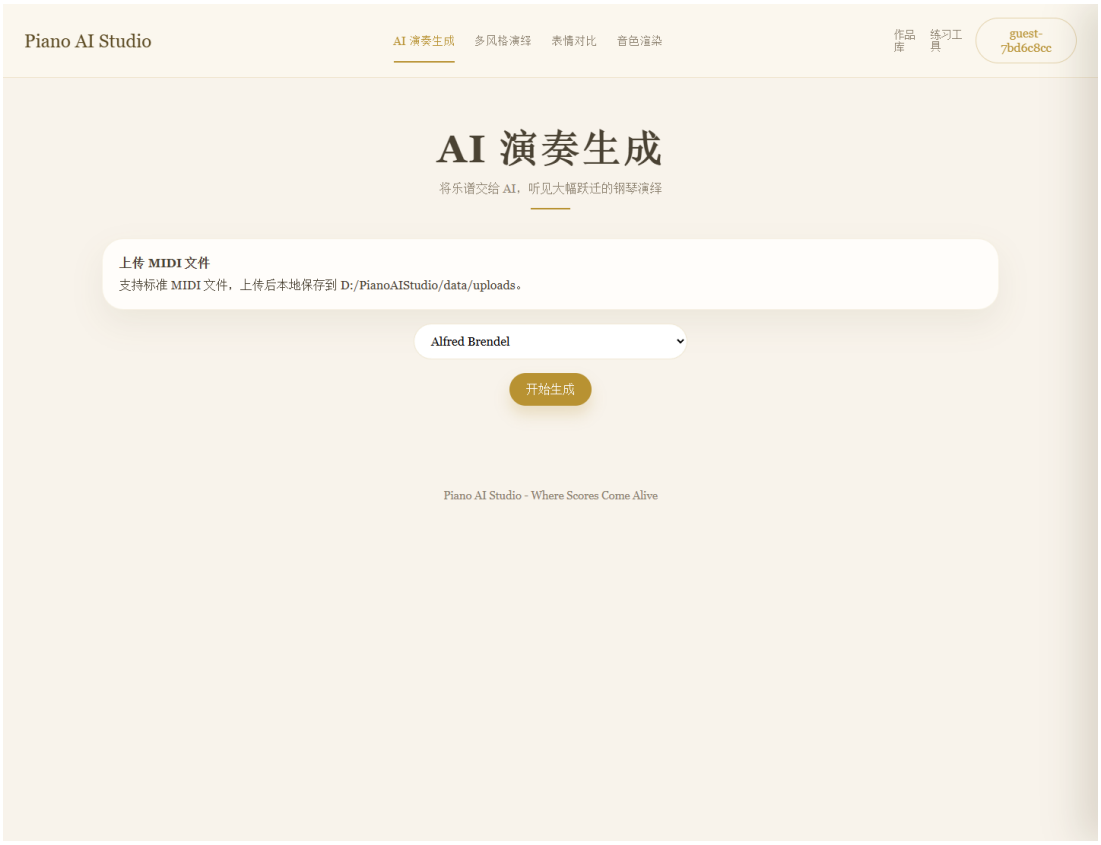

Figure 5-1 AI Performance Generation Entry and Style Selection Interface

5.5 Result Analysis

The test results answer the teacher's question about what the result is. The implemented system successfully transforms mechanical MIDI into expressive MIDI and rendered audio, provides visual comparison charts, stores the generation history, and generates data-grounded analysis. The remaining limitations are that the system still depends on model quality, short ornaments may be over-smoothed, and deeper audio-side evaluation should be added in future work.

The overall test result shows that Piano AI Studio should be understood as an integrated AI music application. Its output is not only an expressive MIDI file. The actual result includes the uploaded work record, the generated expressive MIDI, the rendered WAV audio, the comparison chart, the stored generation history, and the structured analysis report. These elements together make the generation process observable. A user can hear the output, see the parameter changes, revisit the generated record, and download the result for later practice.

From the functional perspective, the strongest result is workflow completeness. The user can start from a local MIDI file and reach a playable, downloadable, and explainable output without leaving the application. This distinguishes the project from a script-level model experiment. A script can prove that a transformation is possible, but it does not by itself provide upload management, progress feedback, result storage, chart comparison, and report explanation. The tests show that the implemented system connects these parts into a usable process.

From the engineering perspective, the main result is that long-running AI work can be wrapped in a stable web interaction pattern. The combination of task records, progress push, file persistence, and front-end state management prevents the user from losing context during generation. This is important for classroom or self-practice use because users are more likely to trust the system when they can see what stage the task is in and where the output file is stored.

From the music-information perspective, the result shows that expressive transformation can be made visible. The system extracts statistics before and after generation, and the comparison page displays differences in velocity and timing. This does not replace professional musical judgement, but it gives learners a concrete basis for noticing expression. For example, a learner can compare whether the AI version introduces stronger

dynamics, whether timing deviation appears in phrase-like positions, and whether the rendered audio matches the visible chart trend.

The tests also show where the current system should be improved. The evaluation scale is still limited because it uses selected MIDI samples and local prototype conditions. A larger evaluation should include more musical styles, different difficulty levels, and a controlled listening procedure. The current listening score is useful for prototype feedback, but it is not enough to prove broad educational effectiveness. This limitation is acceptable for the current graduation project because the main contribution is the complete system design and implementation, not a large-scale music psychology experiment.

Another limitation is that the model output still depends on the quality and style coverage of the expressive generation method. If the input file is too short, too sparse, or rhythmically unusual, the generated expression may not have enough context to shape a convincing phrase. If the input contains dense ornaments, the transformation may smooth details that a human pianist would handle more delicately. These cases do not invalidate the system, but they show that the generated output should be treated as an assisted reference for practice, not as a final performance standard.

The reliability tests indicate that the surrounding software design is as important as the model itself. Even when the model produces a valid output, the user experience would fail if the browser cannot play the audio, the chart cannot load, the download path is wrong, or the history record is missing. Chapter 5 therefore evaluates the system as a whole. The project result is the combination of model transformation and reliable product workflow.

In summary, the expanded testing and experiment results support three conclusions. First, the implemented system can complete the end-to-end workflow from MIDI upload to expressive audio and analysis. Second, the output contains measurable changes in velocity and timing that can be inspected through the comparison interface. Third, the engineering design around progress, storage, and visualization makes the AI generation process usable for learning and practice scenarios. These conclusions are narrower than claiming human-level performance, but they accurately describe what the implemented work achieves.

6. Conclusion and Future Work

6.1 Summary

This thesis designs and implements Piano AI Studio, an AI-assisted piano expressive performance generation system. The work decomposes the problem into MIDI expression

transformation, audio rendering, web service orchestration, interface visualization, storage, and analysis. The final system provides a complete workflow rather than a detached model demonstration.

All code listings that were previously located in Chapter 4 have been moved to the final Appendix. The body chapters now explain the design logic, implementation decisions, difficulties, solutions, and results in prose, while Appendix code provides implementation evidence without interrupting the thesis narrative.

6.2 Limitations

The current system focuses on MIDI-based expressive performance and does not yet evaluate every possible style, genre, or performance tradition. The M2A rendering chain can still be improved for timbre realism. The LLM analysis module is constrained by structured feature input, which reduces unsupported claims but also limits deep musical interpretation.

6.3 Future Work

Future work can improve three directions. First, the model layer can use larger expressive performance datasets and style-conditioned training. Second, the evaluation layer can introduce audio-to-MIDI alignment, onset-level metrics, and controlled listening studies. Third, the learning layer can integrate adaptive piano tutoring, mixed reality feedback, and longer-term practice history.

References

[1] Friberg, A., Bresin, R., & Sundberg, J. (2006). Overview of the KTH rule system for musical performance. Advances in Cognitive Psychology, 2(2-3), 145-161.

[2] Oore, S., Simon, I., Dieleman, S., Eck, D., & Simonyan, K. (2020). This time with feeling: Learning expressive musical performance. Neural Computing and Applications, 32, 955-967.

[3] Jeong, D., Kwon, T., Kim, Y., Lee, I., & Nam, J. (2019). Graph neural network for music score data and modeling expressive piano performance. Proceedings of the 36th International Conference on Machine Learning.

[4] Huang, C. Z. A., Vaswani, A., Uszkoreit, J., Shazeer, N., Simon, I., Hawthorne, C., Dai, A. M., Hoffman, M. D., Dinculescu, M., & Eck, D. (2018). Music Transformer. arXiv:1809.04281.

[5] van den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., et al. (2016). WaveNet: A generative model for raw audio. arXiv:1609.03499.

[6] Kong, J., Kim, J., & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. Advances in Neural Information Processing Systems, 33, 17022-17033.

[7] Hawthorne, C., Stasyuk, A., Roberts, A., Simon, I., Huang, C. Z. A., Dieleman, S., Elsen, E., Engel, J., & Eck, D. (2019). Enabling factorized piano music modeling and generation with the MAESTRO dataset. International Conference on Learning Representations.

[8] Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. Proceedings of ISMIR 2010, 637-642.

[9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.

[10] DeepSeek-AI. (2024). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434.

[11] Wang, C., Li, Z., Wen, Z., Ye, T., Wu, H., & Sun, X. (2023). A systematic review of deep learning-based symbolic music generation. ACM Computing Surveys, 56(4), 1-43.

[12] Nakamura, E., Yoshii, K., & Sagayama, S. (2015). Performance error detection and post-processing for fast and accurate symbolic music alignment. Proceedings of ISMIR 2015, 347-353.

[13] Zhang, H., Tang, J., Rafee, S. R. M., Dixon, S., Fazekas, G., & Wiggins, G. A. (2022). ATEPP: A dataset of automatically transcribed expressive piano performance. Proceedings of ISMIR 2022, 446-453.

[14] Hawthorne, C., Elsen, E., Song, J., Roberts, A., Simon, I., Raffel, C., Engel, J., Oore, S., & Eck, D. (2018). Onsets and frames: Dual-objective piano transcription. Proceedings of ISMIR 2018, 50-57.

[15] Engel, J., Hantrakul, L., Gu, C., & Roberts, A. (2020). DDSP: Differentiable digital signal processing. International Conference on Learning Representations.

[16] Gardner, J., Simon, I., Manilow, E., Hawthorne, C., & Engel, J. (2022). MT3: Multi-task multitrack music transcription. International Conference on Learning Representations.

[17] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.

[18] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., et al. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1-38.

[19] Dong, H. W., Hsiao, W. Y., Yang, L. C., & Yang, Y. H. (2018). MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. Proceedings of AAAI 2018.

[20] Huang, Y. S., & Yang, Y. H. (2020). Pop Music Transformer: Beat-based modeling and generation of expressive pop piano compositions. Proceedings of ACM Multimedia 2020.

[21] Raffel, C., & Ellis, D. P. W. (2014). Intuitive analysis, creation and manipulation of MIDI data with pretty_midi. Proceedings of the 15th ISMIR Late Breaking and Demo Papers.

[22] Raffel, C., & Ellis, D. P. W. (2016). Optimizing DTW-based audio-to-MIDI alignment and matching. Proceedings of ICASSP 2016.

[23] Dannenberg, R. B., Sanchez, M., Joseph, A., Capell, P., Joseph, R., & Saul, R. (1990). A computer-based multi-media tutor for beginning piano students. Interface, 19(2-3), 155-173. https://doi.org/10.1080/09298219008570563.

[24] Smoliar, S. W., Waterworth, J. A., & Kellock, P. R. (1995). pianoFORTE: A system for piano education beyond notation literacy. Proceedings of the Third ACM International Conference on Multimedia, 457-465. https://doi.org/10.1145/217279.215310.

[25] Angelides, M. C., & Tong, X. (1995). Implementing multiple tutoring strategies in an intelligent tutoring system for music learning. Journal of Information Technology, 10(1), 52-62. https://doi.org/10.1177/026839629501000107.

[26] Percival, G., Wang, Y., & Tzanetakis, G. (2007). Effective use of multimedia for computer-assisted musical instrument tutoring. Proceedings of the International Workshop on Educational Multimedia and Multimedia Education, 67-76. https://doi.org/10.1145/1290144.1290156.

[27] Rho, E., Hwang, E., & Kim, H. (2014). Automatic piano tutoring system using consumer-level depth camera. IEEE International Conference on Consumer Electronics, 83-84. https://doi.org/10.1109/ICCE.2014.6775884.

[28] Rogers, K., Rohlig, A., Weing, M., Gugenheimer, J., Konings, B., Klepsch, M., Schaub, F., Rukzio, E., Seufert, T., & Weber, M. (2014). P.I.A.N.O.: Faster piano learning with interactive projection. Proceedings of the Ninth ACM International Conference on Interactive Tabletops and Surfaces, 149-158. https://doi.org/10.1145/2669485.2669514.

[29] Yuksel, B. F., Oleson, K. B., Harrison, L., Peck, E. M., Afergan, D., Chang, R., & Jacob, R. J. K. (2016). Learn Piano with BACh: An adaptive learning interface that adjusts task difficulty based on brain state. Proceedings of CHI 2016, 5372-5384. https://doi.org/10.1145/2858036.2858388.

[30] Molloy, D., Huang, G., & Wunsche, B. C. (2019). Mixed Reality Piano Tutor: A gamified piano practice environment. International Conference on Electronics, Information, and Communication. https://doi.org/10.23919/ELINFOCOM.2019.8706474.

[31] Rigby, J. M., Wunsche, B. C., & Shaw, A. (2020). piARno: An augmented reality piano tutor. 32nd Australian Conference on Human-Computer Interaction, 481-491. https://doi.org/10.1145/3441000.3441039.

[32] Molero, D., Saez-Sobrino, S., Glez-Morcillo, C., Vallejo, D., & Albusac, J. A. (2021). A novel approach to learning music and piano based on mixed reality and gamification. Multimedia Tools and Applications, 80, 16565-16583. https://doi.org/10.1007/s11042-020-09678-9.

[33] Wilson, A. D., & Pfeiffer, T. (2023). Feedback in augmented and virtual reality piano tutoring systems: A mini review. Frontiers in Virtual Reality, 4, 1207397. https://doi.org/10.3389/frvir.2023.1207397.

[34] Amm, S., Mouromtsev, D., Sun, J., & Malaka, R. (2024). Mixed reality strategies for piano education. Frontiers in Virtual Reality, 5, 1397154. https://doi.org/10.3389/frvir.2024.1397154.

[35] Donahue, C., Simon, I., & Dieleman, S. (2019). Piano Genie. Proceedings of the 24th International Conference on Intelligent User Interfaces, 160-164. https://doi.org/10.1145/3301275.3302288.

[36] Benetos, E., Dixon, S., Duan, Z., & Ewert, S. (2019). Automatic music transcription: An overview. IEEE Signal Processing Magazine, 36(1), 20-30. https://doi.org/10.1109/MSP.2018.2869928.

[37] Schedl, M., Gomez, E., & Urbano, J. (2014). Music information retrieval: Recent developments and applications. Foundations and Trends in Information Retrieval, 8(2-3), 127-261. https://doi.org/10.1561/1500000042.

[38] Bello, J. P., Daudet, L., Abdallah, S., Duxbury, C., Davies, M., & Sandler, M. B. (2005). A tutorial on onset detection in music signals. IEEE Transactions on Speech and Audio Processing, 13(5), 1035-1047. https://doi.org/10.1109/TSA.2005.851998.

[39] Muller, M. (2015). Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications. Springer. https://doi.org/10.1007/978-3-319-21945-5.

[40] McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). librosa: Audio and music signal analysis in Python. Proceedings of the 14th Python in Science Conference, 18-25. https://doi.org/10.25080/Majora-7b98e3ed-003.

[41] Raffel, C., McFee, B., Humphrey, E. J., Salamon, J., Nieto, O., Liang, D., Ellis, D. P. W., & Raffel, C. C. (2014). mir_eval: A transparent implementation of common MIR metrics. Proceedings of ISMIR 2014, 367-372.

[42] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

Appendix

Code1: M2M expression parameter mapping and output guarding

```
def apply style(
    midi: pretty midi.PrettyMIDI,
    style preset: str,
) -> pretty midi.PrettyMIDI:
    preset = get preset(style preset)
    notes = [
        note
        for instrument in midi.instruments
        for note in instrument.notes
    ]
    duration = max(midi.get_end_time(), 1.0)
    if not notes:
        return midi

    for index, note in enumerate(sorted(notes, key=lambda item: item.start)):
        phase = note.start / duration
        curve = math.sin(
            (phase * math.pi * 2 * preset["phraseArc"]) + (note.pitch * 0.07)
        )
        rubato = (curve + math.sin(index * 0.4)) * preset["rubatoMs"] / 1000.0
        articulation = preset["articulation"] * 0.12
        new start = clamp(
            note.start + rubato * 0.45,
            0.0,
            max(duration - 0.01, 0.0),
        )
        new end = max(new start + 0.04, note.end + rubato * 0.65 + articulation)
        velocity = (
            note.velocity
            + preset["velocityBias"]
            + curve * 12 * preset["dynamicRange"]
        )
        if note.pitch < 55:
            velocity += preset["warmth"] * 6
        if note.pitch > 75:
            velocity += preset["dynamicRange"] * 2
        note.start = new start
        note.end = new_end
        note.velocity = int(clamp(round(velocity), 20, 126))

    pedal_value = int(20 + preset["pedalAmount"] * 100)
    for instrument in midi.instruments:
        instrument.control changes = [cc for cc in instrument.control changes if
cc.number != 64]
        pedal steps = max(3, int(duration // 2) + 1)
        for step in range(pedal_steps):
            moment = min(duration, step * duration / pedal steps)
            lift = min(duration, moment + 0.9 + preset["pedalAmount"])
            instrument.control changes.append(
                pretty midi.ControlChange(number=64, value=pedal value, time=moment)
            )
            instrument.control changes.append(
                pretty midi.ControlChange(number=64, value=0, time=lift)
            )
    return midi
```

Code2: Backend progress broadcast and frontend subscription logic

```
class ProgressHub:
    def   init  (self) -> None:
        self. clients: dict[str, set[WebSocket]] = {}
        self. lock = asyncio.Lock()

    async def connect(self, task_id: str, websocket: WebSocket) -> None:
        await websocket.accept()
        async with self. lock:
            self. clients.setdefault(task id, set()).add(websocket)

    async def disconnect(self, task id: str, websocket: WebSocket) -> None:
        async with self. lock:
            clients = self._clients.get(task_id)
```

```
            if not clients:
                return
            clients.discard(websocket)
            if not clients:
                self. clients.pop(task id, None)

    async def publish(self, snapshot: TaskSnapshot) -> None:
        payload = snapshot.model dump(by alias=True)
        async with self. lock:
            clients = list(self._clients.get(snapshot.id, set()))
        for websocket in clients:
            await websocket.send json(payload)
```

```
export function subscribeTask(
    taskId: string,
    onMessage: (snapshot: TaskSnapshot) => void,
): () => void {
    const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
    const wsUrl = `${protocol}//${window.location.host}/ws/progress/${taskId}`;
    const ws = new WebSocket(wsUrl);
    ws.onmessage = (event) => onMessage(JSON.parse(event.data) as TaskSnapshot);
    return () => {
        if (
            ws.readyState === WebSocket.OPEN ||
            ws.readyState === WebSocket.CONNECTING
        ) {
            ws.close();
        }
    };
}
```

Code3: Browser-side MIDI parsing and comparison sequence extraction

```
export async function parseMidiFile(
    file: File,
): Promise<ParsedMidiSummary> {
    const arrayBuffer = await file.arrayBuffer();
    const midi = new Midi(arrayBuffer);
    const notes = midi.tracks.flatMap((track) => track.notes);
    if (!notes.length) {
        return {
            noteCount: 0,
            duration: 0,
            pitchRange: [0, 0],
            velocitySeries: [],
            timingSeries: [],
        };
    }

    const pitchRange: [number, number] = [
        Math.min(...notes.map((note) => note.midi)),
        Math.max(...notes.map((note) => note.midi)),
    ];
    const velocitySeries = notes.slice(0, 500).map((note, index) => ({
        x: index,
        y: Math.round(note.velocity * 127),
    }));
    const timingSeries = notes.slice(1, 500).map((note, index) => {
        const previous = notes[index];
        const deviation =
            note.time - previous.time - (note.durationTicks / 960) * 0.5;
        return { x: note.time, y: deviation * 1000 };
    });
    return {
        noteCount: notes.length,
        duration: midi.duration,
        pitchRange,
        velocitySeries,
        timingSeries,
    };
}
```

Code4: Statistical scoring and analysis report section rendering

```
def score difficulty(stats: MidiStats) -> int:
    score = 0.0
    score += min(stats.density * 8.5, 30)
```

```
    score += min(stats.pitch span * 0.35, 18)
    score += min(stats.velocity_std * 0.6, 18)
    score += min(stats.chord density * 5.5, 14)
    score += min(stats.pedal ratio * 28, 10)
    score += min(stats.timing variance * 50000, 10)
    return int(round(min(score, 100)))


export function StatsPanel({ stats }: { stats?: MidiStats | null }) {
    if (!stats) {
        return null;
    }
    const items = [
        ["Total notes", stats.noteCount],
        ["Duration", `${stats.durationSeconds.toFixed(1)} s`],
        ["Average velocity", stats.avgVelocity.toFixed(1)],
        ["Velocity variation", stats.velocityStd.toFixed(1)],
        ["Average tempo", `${stats.avgTempo.toFixed(1)} BPM`],
        ["Pedal ratio", `${(stats.pedalRatio * 100).toFixed(1)}%`],
        ["Pitch span", `${stats.pitchSpan}`],
        ["Chord density", stats.chordDensity.toFixed(2)],
    ];
    return (
        <div className="panel-grid">
            {items.map(([label, value]) => (
                <article key={label} className="metric-card">
                    <span>{label}</span>
                    <strong>{value}</strong>
                </article>
            ))}
        </div>
    );
}
```